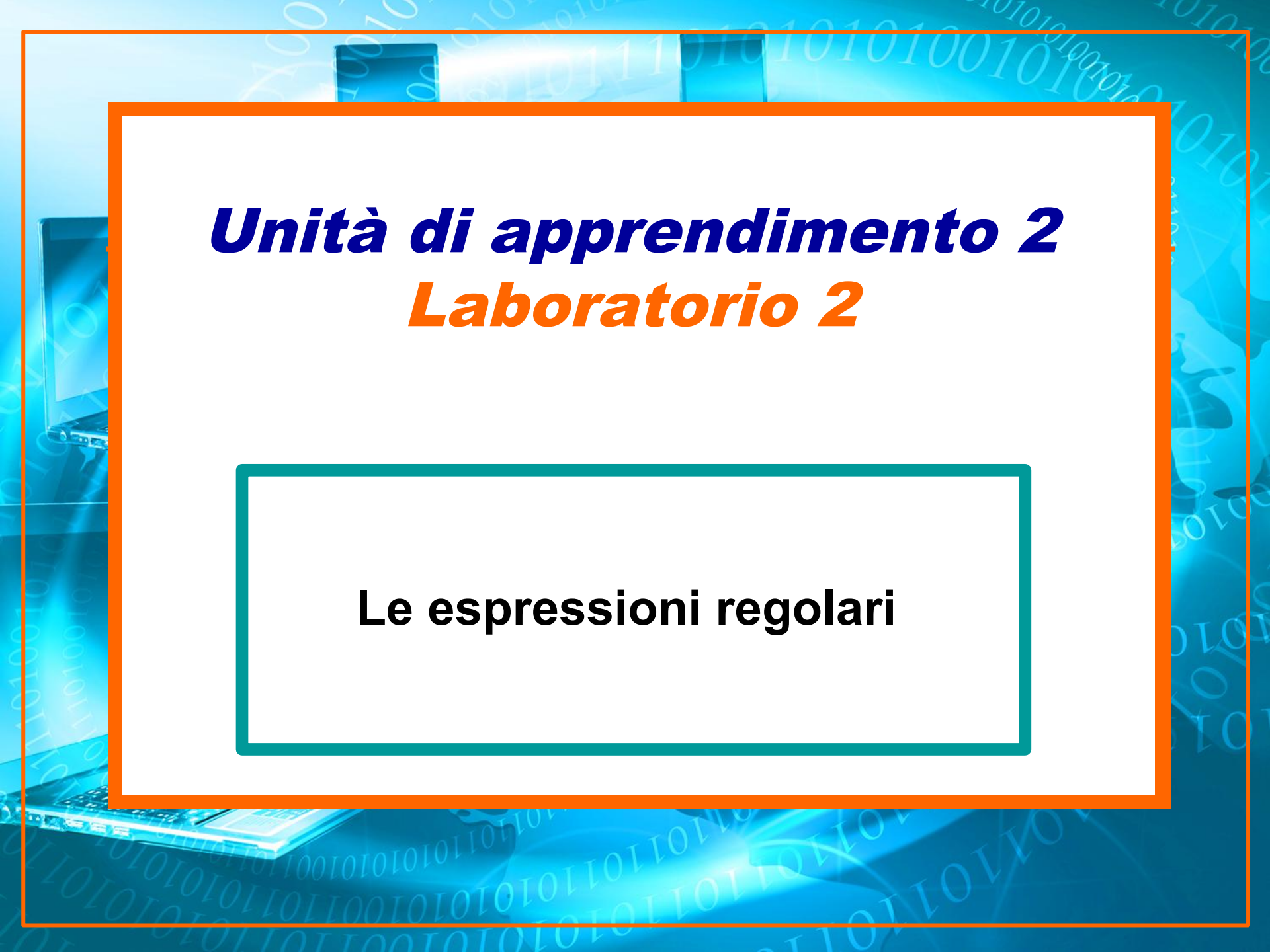


The background of the slide is a blue gradient with a pattern of binary code (0s and 1s) floating around. On the left side, there is a partial view of a laptop screen and keyboard.

Unità di apprendimento 2

**Architettura di rete e formati
per lo scambio dei dati**

The background of the slide is a blue gradient with a pattern of binary code (0s and 1s) floating around. On the left side, there is a partial view of a laptop screen and keyboard.

Unità di apprendimento 2

Laboratorio 2

Le espressioni regolari

Gli automi

- Una lavatrice, un distributore di bevande automatico, un ascensore, un robot per la cucina, un bancomat sono effettivamente degli **automi**
- Anche il computer è un automa, in questo caso programmato, che può tuttavia modificare la funzione svolta a seconda del programma che viene mandato in esecuzione

Gli automi nella vita di tutti i giorni: automa ascensore

- Nell'ascensore lo **stato** è rappresentato dalla situazione nella quale si trova, quindi sale, scende oppure è fermo
- Gli **ingressi** sono i pulsanti della richiesta fornita dall'utente, mentre l'**uscita** è rappresentata dal motore in grado di far salire o scendere l'ascensore stesso



Gli automi nella vita di tutti i giorni: automa distributore bibite/merendine (1)

- **Ingresso** monete: 1, 2, 5, 10, 20, 50 cent, 1 e 2 euro (cardinalità 8)
- **Ingresso** pulsantiera: cardinalità uguale al numero di prodotti erogabili
- **Uscita**: prodotto erogato, con cardinalità data dal numero di prodotti distribuiti



Gli automi nella vita di tutti i giorni: automa distributore bibite/merendine (2)

- Non è detto che basti una moneta per prelevare un prodotto
- Dunque è necessario introdurre il concetto di “memoria” che ricordi le monete inserite, e verifichi se l'importo è sufficiente a erogare la bibita
- Questo concetto è espresso mediante lo **stato di un automa**



Automi a stati finiti

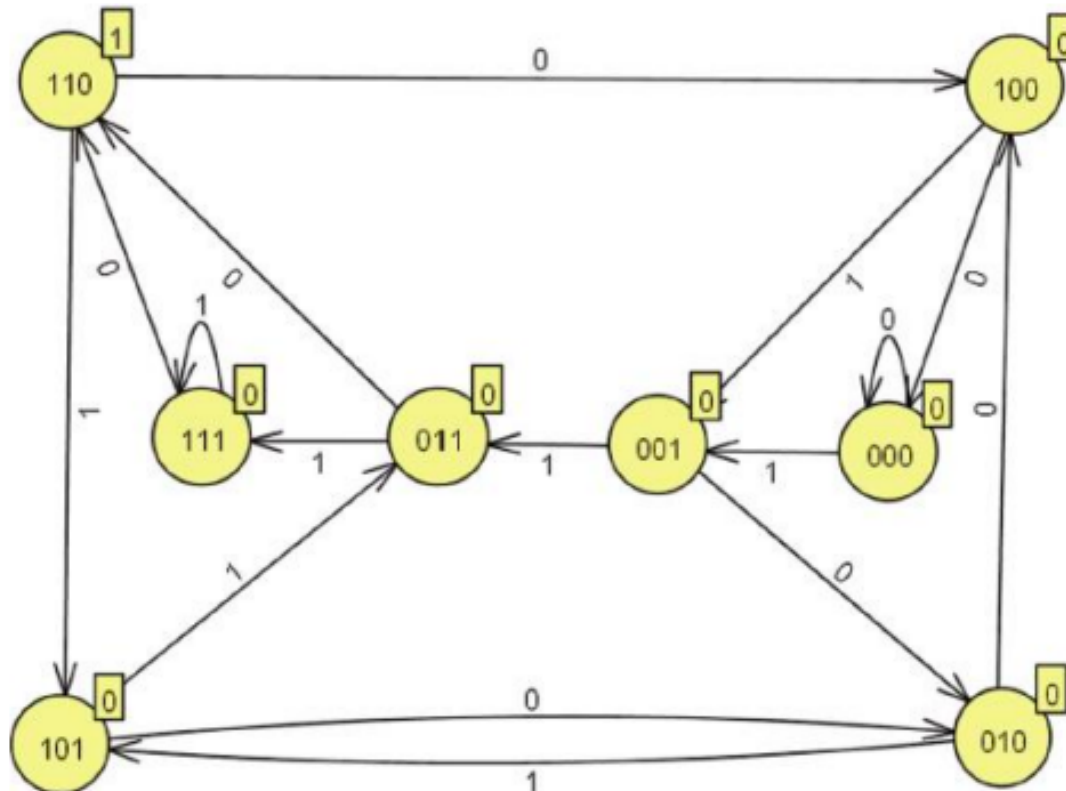
- Un **FA** (Finite Automation) è una **macchina a stati finiti**, ovvero un modello matematico astratto che rappresenta un sistema in grado di trovarsi in un **numero finito di stati** e di passare da uno stato all'altro in base a **simboli di input**
- In altre parole rappresenta un sistema capace di trovarsi in diverse situazioni, dette **stati**, e di cambiare stato **in base a ciò che succede** (un evento o un segnale in ingresso)
- Puoi immaginarlo come una macchina con un numero limitato di modalità di funzionamento, ogni volta che riceve un comando o un segnale, la macchina **passa** da uno stato all'altro seguendo regole precise

Automi riconoscitori

- Gli **automati riconoscitori** sono automi in grado di riconoscere o meno una **sequenza** di valori in ingresso chiamato **pattern**
- Dopo aver ricevuto l'ultimo simbolo della sequenza di input restituiscono una risposta di tipo **true/false** in base all'esito della verifica della sequenza stessa ricevuta
- Lo **stato** indica l'avvenuto **riconoscimento** di una parte della sequenza cercata
- In parole semplici è una macchina che **legge** una stringa, simbolo per simbolo, e decide se accettarla (cioè se la stringa appartiene al linguaggio definito) o rifiutarla

Automi riconoscitori: esempio

- Riconoscitore di sequenza a 3 bit: 110
- Diagramma degli stati:



Espressioni regolari (1)

- Le **espressioni regolari** (ER) permettono di definire in maniera formale un **pattern** (**schema**) di stringhe
- Quindi una ER **describe** le parole da riconoscere, mentre un FA legge le parole e decide se appartengono al linguaggio (**linguaggio regolare**)
- Le ER sono utilizzate principalmente per:
 - specificare pattern di stringhe da **ricercare** all'interno di testi
 - definire le regole di **tokenizzazione** (il processo con cui un testo viene suddiviso in parti più piccole, chiamate token)
- Verificare se una stringa corrisponde a un pattern viene detto **pattern matching**

Espressioni regolari (2)

- L'utilizzo delle ER presenta i seguenti vantaggi:
 - offrono una notazione compatta
 - sono facilmente intelleggibili
 - offrono un metodo per costruire o riconoscere le parole del linguaggio
- Le ER vengono utilizzate in diversi ambienti:
 - linguaggi di programmazione (PHP, Python, Java, C#)
 - shell (DOS, bash)
 - editor di testo (Vi, Emacs)
 - file di configurazione (apache)

Espressioni regolari (3)

- Le ER sono state introdotte da KLEENE negli anni '50, poi utilizzate per la prima volta da Thompson, negli anni '70 in alcuni comandi di gestione di testi in un editor in Unix (ad es. **grep** “Global search for Regular Expression and Print”)
- *Ma dove e quando possiamo utilizzare una ER?!*
- Ogni ER **descrive** un modello che i programmi utilizzeranno per **cercare** nei dati le stringhe che **corrispondono** a tale modello (e eventualmente sostituire le parti di testo trovate)
- Esempi:
 - analisi di log
 - sostituzione di stringhe di testo
 - ricerca in database

Espressioni regolari: alfabeto

- Un'espressione regolare può essere suddivisa in:
 - **caratteri ordinari**: in questo caso abbiamo una **corrispondenza diretta** nelle stringhe, sono i simboli che vogliamo trovare così come sono nelle stringhe
 - **metacaratteri**: **caratteri speciali** che non rappresentano se stessi, ma assumono un significato particolare nelle ER
Lista dei metacaratteri: `[]\{\} | ( ) ^ ? + * .`
 - Vedremo più avanti come si può esprimere il carattere stesso e non il metacarattere

Espressioni regolari: elementi di base

- **Atomo**: un carattere ordinario, un gruppo di caratteri delimitato da [] o una ER fra parentesi tonde (quindi una qualunque ER racchiusa da () diventa un atomo)
 - Sono atomi: a , $[abc]$, (abc) , $(a^*b+c?)$, ...
 - I token invece sono pezzi di testo riconosciuti da una ER
- **Parte**: un atomo eventualmente seguito da un quantificatore
 - Sono parti: a , a^* , $a\{5,6\}$, (a^*b+) , $(a^*b+)?$

Espressioni regolari: espressione banale

- **Espressione banale**: è la forma più semplice (e quindi più banale) di ER
- È costituita solo da caratteri ordinari (dunque senza metacaratteri!)
- Identifica un insieme di stringhe composto da un solo elemento: la stringa **uguale** all'espressione stessa
- Esempi:
 - l'ER **banana** identifica solo la stringa "banana"
 - l'ER **abc** riconosce solo la stringa "abc"

Espressioni regolari: quantificatori

- Un **quantificatore** vincola l'atomo che lo **precede** a comparire tante volte quanto indicato
- Posti m e n due quantificatori generici, avremo:
 - $\{n\}$: esattamente n volte
 - $\{m,n\}$: almeno m e al più n volte => quindi: $\{3,7\}$ significa da 3 a 7 volte
 - $\{0,n\}$: al più n volte
 - $\{m,\}$: almeno m volte
- Sono previste anche forme abbreviate:
 - $*$ equivale a $\{0,\}$ => minimo 0, massimo indefinito ovvero qualunque
 - $+$ equivale a $\{1,\}$
 - $?$ equivale a $\{0,1\}$
- **Attenzione!** Il quantificatore $\{1\}$ si può omettere

Espressioni regolari: esempi con quantificatori

- $a^*b+c?$
 - Interpretazione: 0 o più occorrenze di a, seguite da 1 o più b seguite da 0 o 1 occorrenza di c
 - Stringhe ammissibili: a, ab, abc, bb, abb, abbc, aa
- $Ar(\text{har})\{1,2\}ha$
 - Interpretazione: "Ar" seguito da 1 o due occorrenze di "har" seguita da "ha"
 - Stringhe ammissibili: Arharha, Arharharha
- **Attenzione!**
 - Se scriviamo ab^+ il quantificatore agisce solo su 'b'
 - Se vogliamo applicarlo ad "ab" scriveremo $(ab)^+$
 - Le ER, di norma, sono **case sensitive!**

Espressioni regolari: gruppi di caratteri (1)

- L'atomo **[caratteri]** definisce un insieme da cui possono essere scelti tanti caratteri quanti indicati dal quantificatore applicato (**elenco**)
- Esempio: **[ciao]{4}**
 - Interpretazione: tutte le stringhe lunghe esattamente 4 e composte con i 4 caratteri: c, i, a, o
 - Stringhe ammissibili: ciao, oaic, iaoc, ...
- **[c1-c2]** indica un insieme composto da un **intervallo** di caratteri compreso tra c1 e c2
 - Esempio: **[A-Z]{1,10}** individua tutte le “parole” da 1 a 10 lettere componibili con le lettere maiuscole
- È possibile concatenare intervalli
 - Esempio: **[A-Za-z]** individua tutte le lettere dell'alfabeto

Espressioni regolari: gruppi di caratteri (2)

- Con **^** è possibile **negare** un gruppo di caratteri (**complemento**) ovvero richiedere che il carattere non sia fra quelli indicati [**^elenco-caratteri**] oppure [**^intervallo**]
- Esempio: [**^A-Z**]{5} individua tutte le “parole” di 5 caratteri che non contengono le lettere maiuscole
- Esistono abbreviazioni che rappresentano gruppi di caratteri predefiniti (**classi**):
 - **\d**: qualsiasi cifra decimale (equivale a [**0-9**])
 - **\D**: qualsiasi carattere escluse le cifre (equivale a [**^0-9**])
 - **\w**: qualsiasi carattere alfanumerico, compresi ‘_’ e spazio (equivale a [**a-zA-Z0-9_**])
 - **\W**: qualsiasi carattere non alfanumerico (equivale a [**^a-zA-Z0-9_**])

Espressioni regolari: altri metacaratteri

- `.` corrisponde a un carattere qualsiasi
 - Esempio: `[.]{10}` rappresenta le parole scritte con qualsiasi carattere lunghe 10 caratteri
- `|` separa espressioni regolari ed equivale ad OR
 - Esempi: `abc|def|xyz` individua abc, def e xyz mentre `abc(def|xyz)` individua abcdef e abcxyz
- `\` consente di inserire un metacarattere come carattere ordinario (`\` si dice **carattere di escape**)
 - Esempi: `\.` indica il carattere punto mentre `\(` indica la parentesi aperta
- `^` e `$` sono delle **ancore**, vincolano la posizione della sequenza di caratteri rispettivamente all'inizio o alla fine della stringa
 - Esempio: `.*\.txt$` individua tutte le stringhe che finiscono per .txt

Espressioni regolari: altri esempi (1)

- `[-+]?[0-9]+` → tutti i numeri interi con segno
- `gr(a|e)y` → {'gray', 'grey'}
- `gr[ae]y` → {'gray', 'grey'}
- `a[_ ]b` → {'a_b', 'a b'}
- `a(cb)*d` → {'ad', 'acbd', 'acbcbd', ...}
- `a*\*` → {'*', 'a*', 'aa*', 'aaa*', ...}
- `a\\c` → {'a\\c'}

Espressioni regolari: altri esempi (2)

- $[+-]?17 \rightarrow \{'17', '+17', '-17'\}$
- $1[^0-4] \rightarrow \{'15', '16', '1a', '1T', \dots\}$
- $ab+c \rightarrow \{'abc', 'abbc', 'abbbc', \dots\}$
- $ab?c \rightarrow \{'ac', 'abc'\}$
- $ab^*c \rightarrow \{'ac', 'abc', 'abbc', 'abbbc', \dots\}$
- $1[0-4] \rightarrow \{'10', '11', '12', '13', '14'\}$

Espressioni regolari: esempi d'uso concreti

- In **Python** si include il modulo **re**
 - [documentazione ufficiale](#)
- In **C#** si usa **System.Text.RegularExpressions** e la classe **Regex**
 - [documentazione ufficiale](#)
- In **Java** si importa **java.util.regex** e le classi **Pattern** e **Matcher**
 - [documentazione ufficiale](#)
- In **PHP** abbiamo a disposizione diverse funzioni tra cui **preg_match()**, **preg_match_all()**, **preg_replace()**
 - [documentazione ufficiale](#)
- In **MySQL** abbiamo a disposizione diverse funzioni tra cui **REGEXP()**, **REGEXP_REPLACE()**, **REGEXP_LIKE()**
 - [documentazione ufficiale](#)

Espressioni regolari: riferimenti

- Tutorial: <https://www.regular-expressions.info>
- Esercitarsi online: <https://regexr.com>
- Fonti
 - <https://drive.google.com/file/d/1uur-AFMzPqew4pxiX1cB6VuqPswlLZUs/view?usp=sharing>
 - <http://lia.deis.unibo.it/Courses/TecnologieWeb0910/lezioni/3.03.RegExp.pdf>
 - https://homes.di.unimi.it/ferrari/FondInfoSic2008_09/reg_exp_doppio.pdf
 - https://www.dieei.unict.it/sites/default/files/documenti_sito/python-lezione-3.pdf